Live Looping Sampler

Documentation ::: Sync/Tempo

SooperLooper can quantize most operations with a choice of several sources and parameters. When enabled, the quantize operations affect all loops.



The **sync to** control chooses what source to use for synchronization: None, Internal Tempo, MIDI clock, JACK transport, or any of the existing loops.

The **tempo** control shows the current tempo, and is only editable directly when **sync to** is set to None or Internal. When MIDI clock is selected, the tempo will be updated to reflect the estimated tempo based on any incoming MIDI clock ticks. When JACK transport is selected, it will always reflect the tempo specified by the current JACK time master (if any). However, when one of the loops is selected as the sync source, the tempo is defined entirely by the current loop's length and the **8th/cycle** parameter.

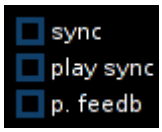
The **tap** button lets you tap in a tempo when in Internal (or None) sync-to mode. This function can also be done with a keybinding ('t' by default) and a MIDI binding. The tap button also pulses with the current tempo giving a visual indication of it.

The **8th/cycle** parameter defines how many 8th notes as defined by the current tempo (there are 2 eighths per beat) are in a cycle of a current loop, or a potential loop. As described below, the cycle length is useful as a quantization boundary, as well as an eighth-note itself.

The **quantize** parameter defines when operations sync to in reference to the sync source. The available choices are Off, Cycle, 8th, and Loop. Note that no sync will ever occur if this value is set to Off. When the sync source is another loop, the choices are all valid.

The **rel sync** parameter specifies a special mode of sync when recording new loops similar to the EDP's unquantized SyncRecord function. When used in conjunction with a sync source, a quantize setting other than off, and a loop's "sync" option, will allow you to start recording a new loop at any time and it will keep track of the sync and round off the loop so the length is a multiple of the sync interval. In other words this allows you to be

synced to another loop, midi, etc but start recording exactly when you want (instead of it waiting until the next sync point) and still get the right length loop.



If the **sync** option is checked for a particular loop, operations will be quantized to the selected boundary. This includes Record, Multiply, Replace, Substitute, Insert, Reverse, Trigger, and Once. Note that Overdub is never quantized. When a command is performed, the actual operation won't start/stop until the precise moment of the next sync boundary arrives. For instance, when the sync source is a loop, and the quantize parameter is Cycle, a Record operation will start and stop on an exact cycle boundary of the source loop. More interesting polyrhythms are possible when using 8ths as the quantize parameter, for example.

The **play sync** option for a loop causes the playback of the loop to be coerced to remain in sync with the selected syncto source. Currently, this is done by retriggering the loop if an external sync boundary occurs when near a loop boundary (+/- the current quantize type). This technique is particulary suitable for maintaining playback sync with MIDI clock which may drift with respect to the audio playback. Note that it does **not** currently do any timestretching or rate resampling, so it isn't suitable for dealing with external tempo changes from the original recorded loop's tempo.

Latency Compensation

Latency compensation allows those of you who don't monitor your live input through software have SL put your overdubs in the right place, irregardless of your buffer size. Please note that if you are currently playing software instruments or using SW effects inline for monitoring, then you probably won't use the compensation feature because you are hearing what you get when you play with the loop, and your only real choice is to use small buffer sizes to get low latency. This might not be strictly true with regard to initial loop record/playback and triggering, we'll see.

In the GUI you will see a new tab in the Preferences window called Latency. This exposes controls for Input Latency and Output Latency, as well as a toggle to automatically pick the values for these based on the information gathered from JACK or the AU plugin host. Note the numbers from JACK are more likely to be correct, as AU plugins don't get all the info they need to judge it. Either way, the automatic will only get you close, you might need to tweak the values yourself based on your hardware setup as well. Basically, both controls together affect the compensation of overdubs, but the Input Latency also affects what content gets added to loop. If it is too low, you can actually get audio the was played before you started the operation, and if too high you will cut off audio you intended to get

recorded. Once the Input latency feels right, if the overdubs aren't aligned with the existing loop audio the way you heard it when playing, then adjust the Output Latency. There is an excellent description of this in Mobius' documentation in the latency calibration section. Check out the last 3 paragraphs of that section especially, which I have included here:

If you want to tweak the latency values manually, start by increasing the values for both latencies by 10 until it sounds right. You normally shouldn't have to add more than a few hundred frames. If it is still way out of alignment, stop adding to the input latency and start making more radical adjustments to the output latency, up or down in increments of 500. If that is required, submit a bug report with information about your computer and sound card.

To get a better understanding of what these values mean, you can think that at any moment in time the software is receiving sound that was performed a few milliseconds in the past, and is playing sound that will be heard a few milliseconds in the future. Input latency is the number of frames it takes a sound to "get into" the software. If you set input latency too low, you will be recording sound that was performed before the time you triggered the record function. If you set input latency too high, you will lose some of the sound immediately after you triggered the record function. It is more critical that input latency be correct because miscalibration can result in unwanted sound or loss of sound at the record start point. If an overdub sounds like it has the right content but isn't aligned properly, then adjust output latency not input latency.

If you set output latency too low, overdubs will sound like they are playing too late. If output latency is too high, overdubs will sound like they are playing too early. Deliberately adjusting output latency too low can actually be useful to compensate for monitoring latency. If you perform live using monitor speakers some distance away, it will take a noticeable amount of time for the sound to travel from the speakers to your ears. If you are overdubbing to this sound, it may not be aligned properly on playback. Raising output latency can help bring the overdubs back into alignment.

The Latency tab also has a "Automatically Disable Compensation when Monitoring input" option, which when enabled automatically turns off any compensation when your 'main in mon' control is not -infinity. This enforces the fact that if you are passing the input through for monitoring in SW you are hearing andn playing to the latency already. However, you may still want some input compensation to get the intended content in the loop based on your triggering of the operation.

© 2005 Jesse Chappell *<jesse at essej dot net>*

Documentation
Getting Started
Commands
Sync/Tempo
MIDI Bindings
OSC Interface
MIDI Commands
Donate
Forum
Email List